
Relazione Quake Engine

Redacted

879372

Informatica Grafica

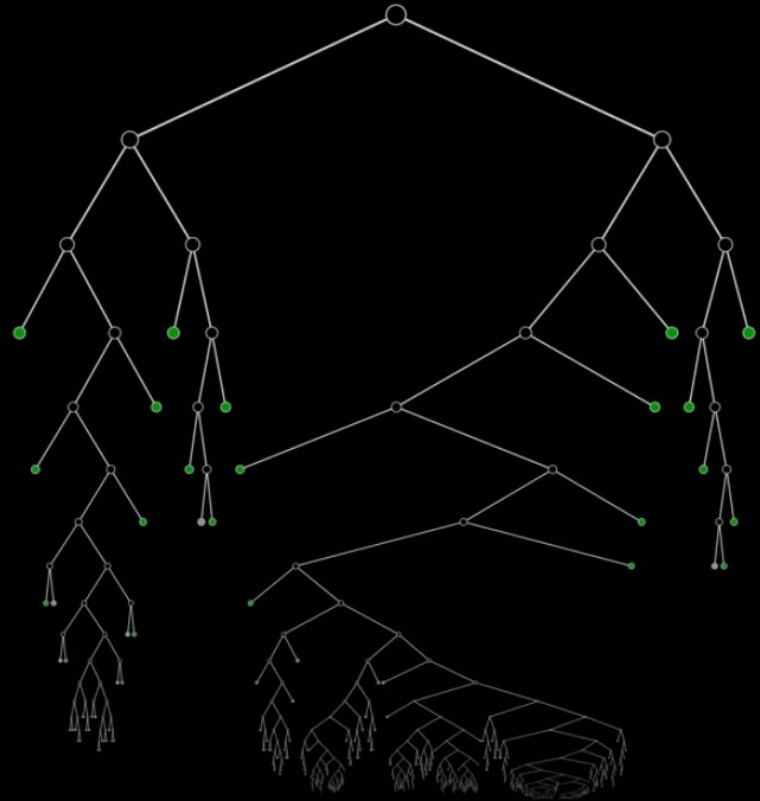
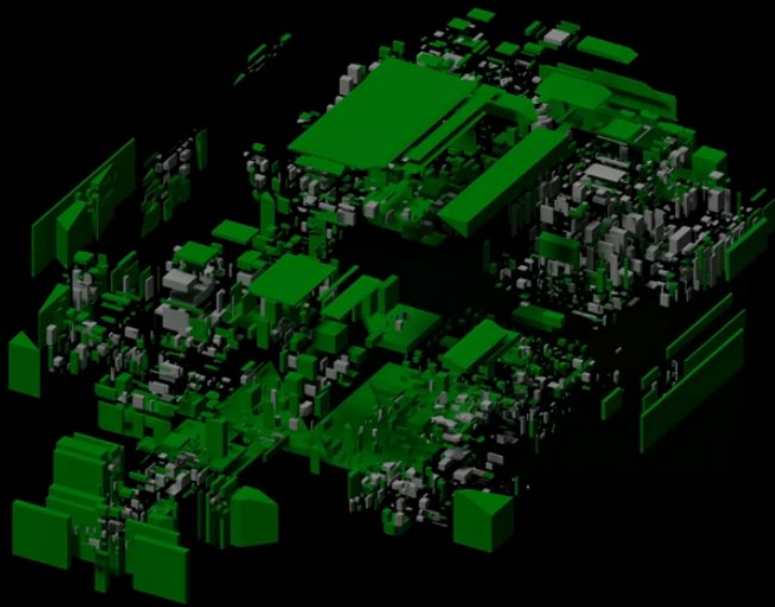
Giugno 2024

Storia

- Nel 1996 Id Software rilascia il gioco *QUAKE* per MS DOS
 - Programmatori: John Carmack, Michael Abrash, John Cash
 - Lead designer: John Romero
- Segue il successo di *DOOM* e *DOOM II*
 - Sparatutto in prima persona più popolare di sempre
 - Grafica 2.5D, "Sprite" che guardano la telecamera
- John Carmack sviluppa un motore "true 3D"
 - Migliori effetti di illuminazione
 - Percezione spaziale realistica, verticalità
 - Rendering più efficiente
 - Difficoltà nella determinazione delle superfici visibili e ottimizzazione

Binary Space Partitioning

- Corridoi, labirinti e stanze chiuse permettono di **dividere lo spazio** con dei piani separatori
 - Lo spazio del mondo è chiuso in un poligono convesso senza "leak"
- **Geometria statica** memorizzata in un albero binario
 - Preso il piano su cui giace un qualunque poligono viene diviso lo spazio in due sottoalberi
 - Se di fronte: Sotto-Albero Sinistro
 - Se si trova dietro: Sotto-Albero Destro
- Genera foglie contenenti sezioni piene o vuote necessariamente **convesse**
 - Quake memorizza i poligoni nelle foglie "Vuote" (empty leaves)
 - Quindi solo le facce che guardano all'interno del poligono
- Visita **in-order** garantisce l'ordine **Back-to-Front**



Matt's Ramblings: "BSP Trees: The Magic Behind Collision Detection in Quake" su Youtube

Algoritmi di creazione e visita

```
Tree make_tree(polygon_list pl) {
    int k = Select_polygon(pl);
    polygon_list pos_list = null; // Sotto-alberi
    polygon_list neg_list = null; //
    for (int i = 0; i < Size_of(pl); i++) {
        if (i != k) {
            polygon pos_parts, neg_parts;
            Split_polygon(pl[i], pos_parts, neg_parts);
            Add(pos_parts, pos_list);
            Add(neg_parts, neg_list);
        }
    }
    return Combine_tree(make_tree(pos_list), pl[k],
                        make_tree(neg_list))
}
```

```
void Back_to_front(viewing_position eye, BSP_tree t) {
    if (t != NULL) {
        if (pos_side_of(t->root, eye)) {
            Back_to_front(eye, t->neg_branch);
            Visit(t->root);
            Back_to_front(eye, t->pos_branch);
        } else {
            Back_to_front(eye, t->pos_branch);
            Visit(t->root);
            Back_to_front(eye, t->neg_branch);
        }
    }
}
```


Algoritmi di creazione e visita

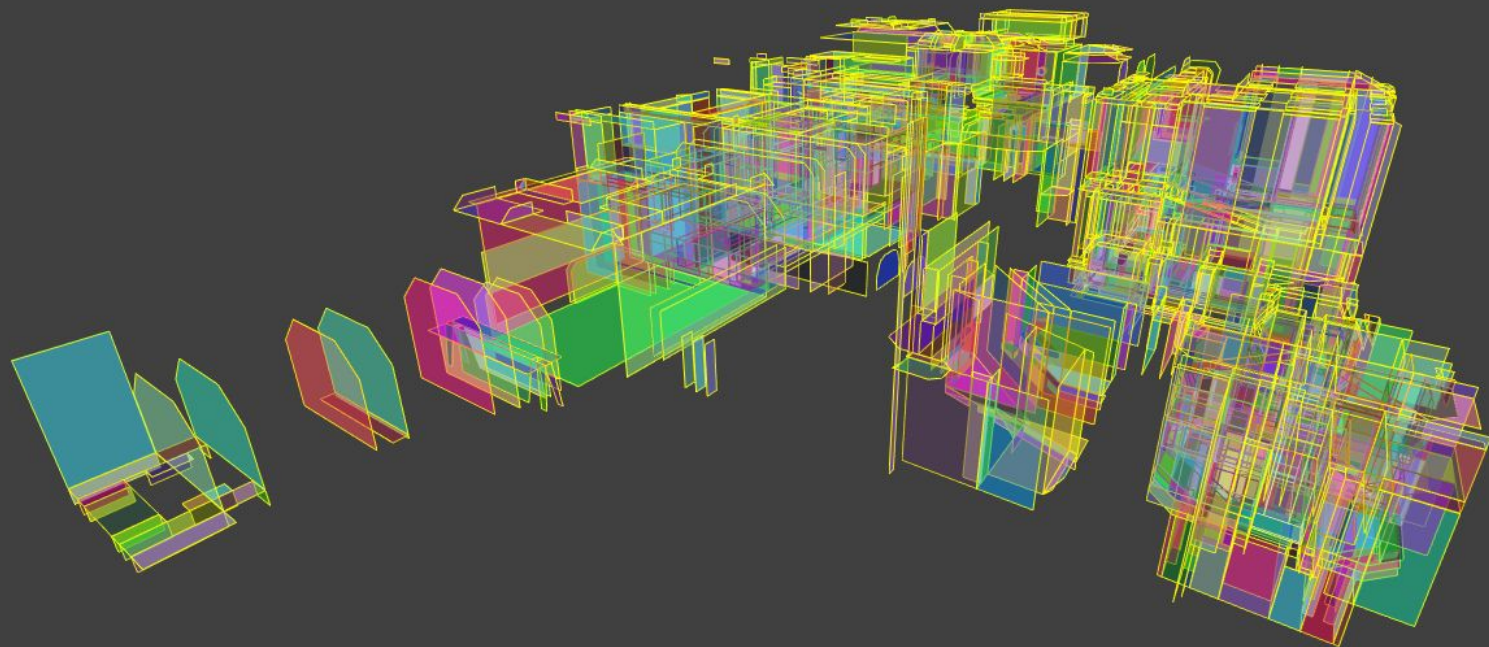
```
Tree make_tree(polygon_list pl) {
    int k = Select_polygon(pl);
    polygon_list pos_list = null; // Sotto-alberi
    polygon_list neg_list = null; //
    for (int i = 0; i < Size_of(pl); i++) {
        if (i != k) {
            polygon pos_parts, neg_parts;
            Split_polygon(pl[i], pos_parts, neg_parts);
            Add(pos_parts, pos_list);
            Add(neg_parts, neg_list);
        }
    }
    return Combine_tree(make_tree(pos_list), pl[k],
                        make_tree(neg_list))
}
```

```
void Back_to_front(viewing_position eye, BSP_tree t) {
    if (t != NULL) {
        if (pos_side_of(t->root, eye)) {
            Back_to_front(eye, t->neg_branch);
            Visit(t->root);
            Back_to_front(eye, t->pos_branch);
        } else {
            Back_to_front(eye, t->pos_branch);
            Visit(t->root);
            Back_to_front(eye, t->neg_branch);
        }
    }
}
```

Questa struttura dati è salvata all'interno di file .BSP per ogni livello di gioco, questi file si trovano insieme agli altri assets in PAK0.pak

Frustum Clipping e PVS

- L'obiettivo è diminuire la geometria da renderizzare
 - Un primo passo consiste nel clippare tutti i nodi dell'albero BSP che risultano fuori dal frustum di vista
- Nella fase di pre-rendering vengono generati dei **Potentially Visible Set** per ogni foglia dell'albero BSP
 - Contengono tutte le foglie visibili da ogni foglia
 - Vengono costruiti a partire da un **Portal Graph**
- I **portali** sono poligoni "vuoti" che dividono una foglia da un'altra
 - L'unico modo per **muoversi da una foglia ad un'altra** è attraverso questi portali
 - Vengono estratti nella fase di compilazione e inseriti in una struttura a grafo
 - Utilizzati per generare le liste di visibilità
 - Vengono individuati da compilatore e memorizzati nel file .BSP



Visualizzazione dei portali sul software NetRadiant

Potentially Visible Sets

- Per ogni **coppia di portali** viene generato un **piano separatore** che parte da un lato del primo portale e passa per un lato **opposto** del secondo



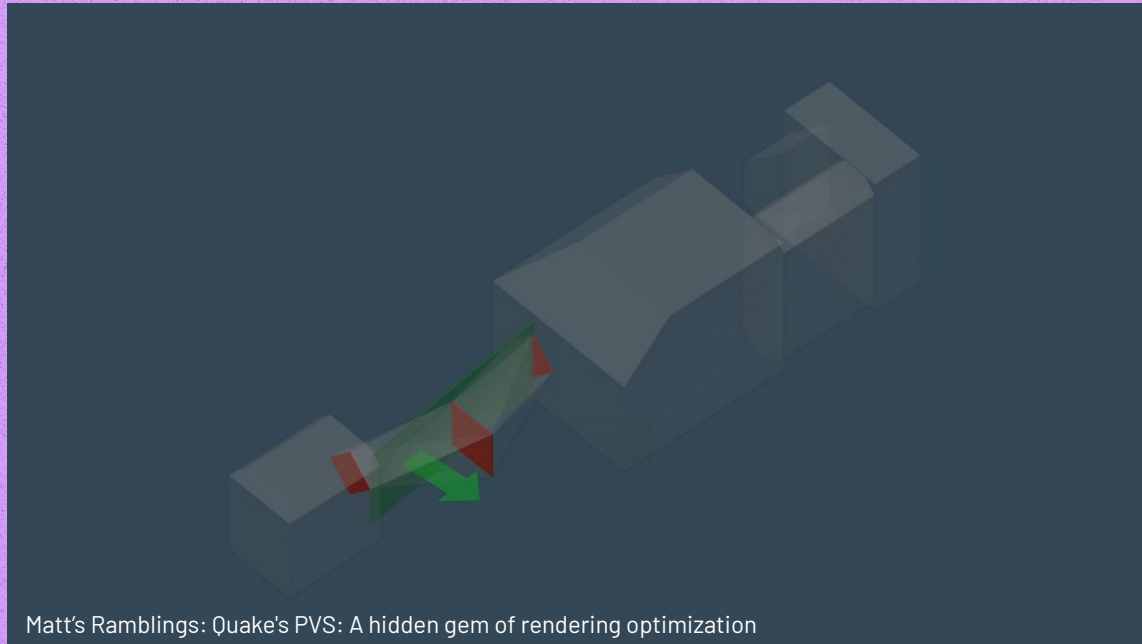
Potentially Visible Sets

- Questo piano viene utilizzato per **clippare** il terzo portale eliminando la sezione che si trova **dietro alla direzione della normale**



Potentially Visible Sets

- Se il portale è **interamente tagliato**, la foglia che collega viene scartata interamente insieme a tutte quelle successive nel grafo non raggiungibili



Potentially Visible Sets

- Se però dobbiamo tenere in considerazione più di tre portali il processo è troppo complicato e oneroso
- Quake utilizza un approccio che consiste nel considerare **tre portali** alla volta
 - Un vettore che passa tra tutti i portali intermedi tra quello di origine e quello di arrivo **passa necessariamente** attraverso il sottoinsieme che ne considera solo tre
 - **Però** ci sono vettori che passano tra i portali presi in considerazione, ma non quelli intermedi
 - Da qui l'avverbio "**Potenzialmente**"
- Questi insiemi sono calcolati per ogni percorso del grafo in fase di precompilazione in maniera ricorsiva tramite una **ricerca depth-first**

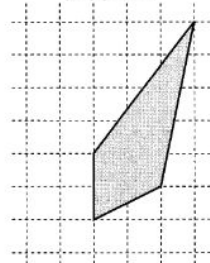
Hidden Surface Removal

- I **PVS** non eliminano completamente l'overdraw
 - Il **Painter's Algorithm** non è abbastanza efficiente
 - Z-Buffering con solo il software richiede troppi sacrifici
- Viene utilizzato lo **span-sorting**
 - L'albero BSP è visitato **Front-To-Back**
 - Utilizza **Active Edge Lists** per ogni poligono nel frustum
 - In un ambiente 3D è **difficile determinare quale è il punto iniziale o finale** di una scanline (span) quindi viene mantenuta anche una **Active Polygon List**
 - Per ogni scanline vengono ordinati sia gli span che i poligoni
 - Gli span vengono clippati tra loro per evitare sovrapposizioni
- Rispetto allo **Z-Buffering è più efficiente** siccome considera interi span (coppie punto di inizio e di fine) invece che singoli pixel

Hidden Surface Removal



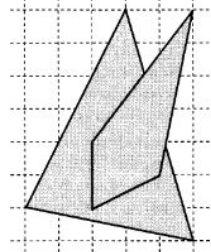
polygon A



spans

$x = 22, y = 0, \text{count} = 0$
$x = 22, y = 1, \text{count} = 0$
$x = 21, y = 2, \text{count} = 1$
$x = 20, y = 3, \text{count} = 2$
$x = 19, y = 4, \text{count} = 3$
$x = 19, y = 5, \text{count} = 2$
$x = 19, y = 6, \text{count} = 0$

A and B composited



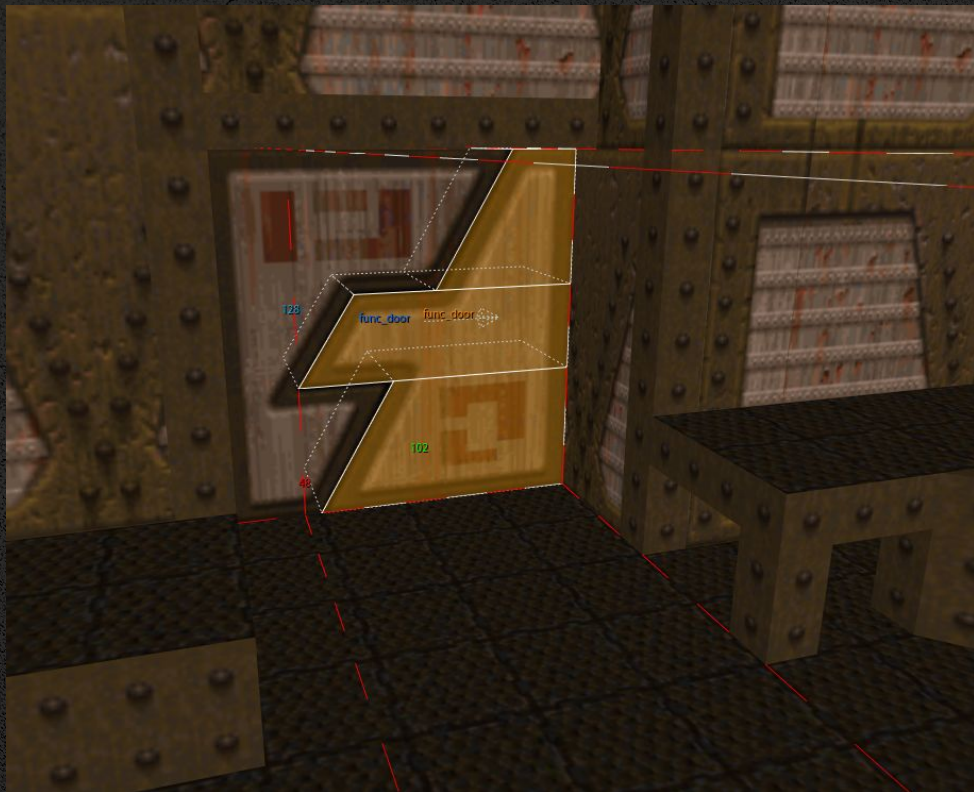
visible spans

A: $x = 20, y = 0, \text{count} = 0$	B: $x = 22, y = 0, \text{count} = 0$	
A: $x = 20, y = 1, \text{count} = 1$	B: $x = 22, y = 1, \text{count} = 0$	
A: $x = 19, y = 2, \text{count} = 2$	B: $x = 21, y = 2, \text{count} = 1$	
A: $x = 19, y = 3, \text{count} = 1$	B: $x = 20, y = 3, \text{count} = 2$	
A: $x = 18, y = 4, \text{count} = 1$	B: $x = 19, y = 4, \text{count} = 3$	
A: $x = 18, y = 5, \text{count} = 1$	B: $x = 19, y = 5, \text{count} = 2$	A: $x = 20, y = 5, \text{count} = 1$
A: $x = 17, y = 6, \text{count} = 5$	B: $x = 19, y = 6, \text{count} = 0$	
A: $x = 22, y = 7, \text{count} = 0$		

Moving Entities

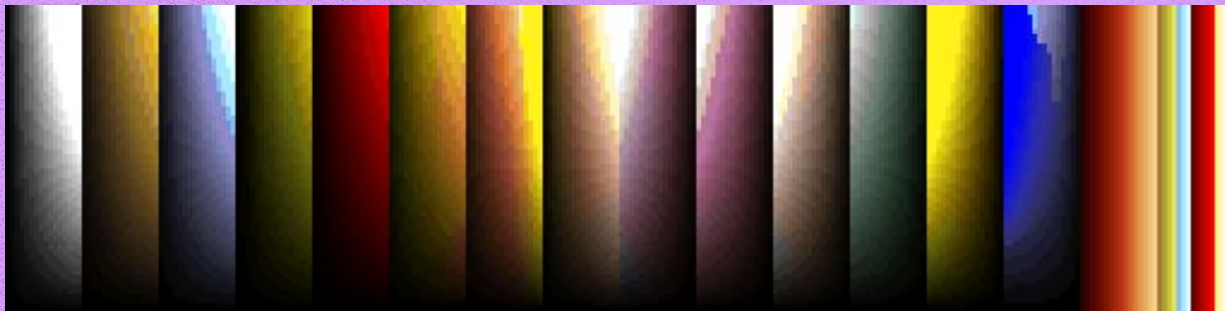
- In Quake esistono tre tipi di entità
 - **Modelli 3D Poligonali (Alias)**
 - **Modelli BSP**
 - **Sprite**
- Per i modelli 3D poligonali viene utilizzato lo **Z-Buffering** siccome si tratta di poche entità (mostri, giocatori e armi)
- Per gli sprite pure (effetti particellari)
- Per i modelli BSP viene utilizzato lo stesso meccanismo delle mappe di gioco, vengono inseriti all'interno dell'albero BSP globale
 - Se sono presenti **più modelli BSP in una foglia** vengono **ordinati** per $1/z$ (calcolata dall'equazione del piano in cui giacciono)

Moving Entities



Texture →

- Le texture in Quake utilizzano una palette predefinita di **256 colori**
 - Tramite una **colormap** viene estratto il colore di un pixel
 - Questo colore tiene conto della componente di **luminosità**
 - Gli indici della matrice rappresentano il colore (0 - 255) e la sua luminosità (0 - 63)
 - La luminosità può andare fino al 200% (**overbrightness**)
 - Esistono pixel sempre con l'illuminazione massima (**fullbright**)



Illuminazione e Shading

- Per l'illuminazione quake usa delle **lightmaps** applicate ad ogni superficie
 - Griglie di valori di illuminazione per ogni **16x16 pixel**
 - Si trovano sempre nel file .BSP precompilato
 - Possibile **sovrapporre** diverse lightmaps anche **dinamicamente**
 - Viene applicato un filtro di **Smoothing** sui bordi per avere un effetto graduale
 - Ogni texel **illuminato in base alla media di questi punti**
- Viene utilizzato il **Gouraud Shading** per le entità
 - Sono illuminate con una luce ambientale e una luce fissa distante in base all'illuminazione della superficie su cui si appoggiano
- Per risparmiare memoria vennero implementati:
 - **Surface Caching**: baking di texture e lightmaps salvati in una cache
 - **MIP-Mapping**: normalizzazione delle texture in base alla distanza dalla camera

Renderer OpenGL

- Primo tentativo, **porting su GPU Rendition Verità** con VQuake
 - Software proprietario e mal documentato (per Carmack)
- QuakeGL, WinQuake e Quake World utilizzano **OpenGL 1.0**
- Viene utilizzato lo **Z-Buffering** per tutti i poligoni
- Usa comunque la CPU per lo shading
 - La pipeline fixed-function utilizza comunque Gouraud
- Lightmaps e Texture gestite tramite il **Multi-Texturing**
 - Permette **lightmaps RGB**
 - Pixel overbright e fullbright non mostrati correttamente
- **Texture filtering** che rende le superfici più smussate
- Introduce una **correzione sul Texture mapping**

References

- Michael Abrash's Graphics Programming Black Book
- Henry Fuchs, Zvi M Kedem, and Bruce F Naylor. On visible surface generation by a priori tree structures.
- Id Software. Quake. <https://github.com/id-Software/Quake>
- David Kushner. Masters of Doom: How two guys created an empire and transformed pop culture.
- Matt's Ramblings www.youtube.com/@MattsRamblings
- Fabien Sanglard <https://fabiansanglard.net/quakeSource/index.php>
- Quaddicted https://www.quaddicted.com/engines/software_vs_glquake